C++ speedrun

Πίνακας περιεχομένων

Πρόλογος **4**
Γιατί C++; . . . 4
Τι εννοούμε «σύγχρονη» C++ . . . 4
Τι χρειάζεται να ξέρετε . . . 4
Πώς να διαβάσετε τις σημειώσεις . . . 5

1 Εγκατάσταση Εργαλείων **6**
1.1 Linux . . . 6
1.1.1 Ubuntu, Debian, Linux Mint . . . 6
1.1.2 Fedora . . . 6
1.1.3 Arch Linux, Manjaro . . . 7
1.2 macOS . . . 7
1.2.1 Βήμα 1: Command Line Tools της Apple . . . 7
1.2.2 Βήμα 2: Homebrew . . . 7
1.2.3 Βήμα 3: GCC και CMake . . . 8
1.3 Windows . . . 8
1.3.1 Βήμα 1: Εγκατάσταση του MSYS2 . . . 8
1.3.2 Βήμα 2: Ενημέρωση του MSYS2 . . . 9
1.3.3 Βήμα 3: Εγκατάσταση των εργαλείων . . . 9
1.3.4 Βήμα 4: Προσθήκη στο PATH . . . 9
1.4 Έλεγχος της εγκατάστασης . . . 9
1.5 Εγκατάσταση IDE . . . 10
1.6 Εγκατάσταση VSCode . . . 10

2 Κατασκευή προγράμματος **12**
2.1 Compilers - Interpreters . . . 12
2.2 Συναρμολόγηση προγράμματος . . . 13

3 Εισαγωγή στην C **16**
3.1 Ένα πρώτο πρόγραμμα . . . 16
3.2 Τύποι αρχείων . . . 16
3.3 Εισαγωγή σχολίων σε κώδικα . . . 17
3.4 Εντολές Προεπεξεργαστή . . . 18
3.4.1 Η εντολή #include . . . 18
3.4.2 Linux / macOS . . . 20

3.4.3 Windows (PowerShell) 20
3.4.4 Η εντολή #define 22

Πρόλογος

Οι σημειώσεις αυτές απευθύνονται σε φοιτητές Φυσικής που θέλουν να μάθουν C++ με σύγχρονο τρόπο και με γνώμονα τις ανάγκες της επιστημονικής δουλειάς, με έμφαση στην κατασκευή ασφαλούς, οργανωμένου και αποδοτικού κώδικα.

Γιατί C++;

Η C++ παραμένει μία από τις βασικές γλώσσες του επιστημονικού υπολογισμού. Σε αυτήν είναι γραμμένα εργαλεία όπως το ROOT και το Geant4 του CERN, και τα τελευταία χρόνια είναι ο βασικός υποψήφιος για υψηλής απόδοσης προσομοιώσεις (High-Performance-Computing, HPC). Συνδυάζει υψηλές επιδόσεις με εργαλεία που επιτρέπουν τη συγγραφή καθαρού, ασφαλούς και επαναχρησιμοποιήσιμου κώδικα.

Τι εννοούμε «σύγχρονη» C++

Από το πρότυπο C++11 και έπειτα η γλώσσα έχει αλλάξει ριζικά. Εδώ τη διδάσκουμε όπως γράφεται σήμερα (C++17 και C++20), καθώς προσφέρει εντολές υψηλότερου επιπέδου σε σχέση με την C, που αυτοματοποιούν ορισμένες διαδικασίες με μηδενικό κόστος στην απόδοση (έξυπνοι δείκτες, `std::vector`, κ.α.).

Τι χρειάζεται να ξέρετε

Δεν απαιτείται προηγούμενη εμπειρία στην C++, ούτε στην C. Μια στοιχειώδης εξοικείωση με κάποια άλλη γλώσσα προγραμματισμού (π.χ. Python) βοηθά, αλλά δεν είναι απαραίτητη. Τα μαθηματικά που χρησιμοποιούνται δεν ξεπερνούν το επίπεδο των πρώτων ετών σπουδών.

Πώς να διαβάσετε τις σημειώσεις

Ο προγραμματισμός μαθαίνεται στην πράξη. Πληκτρολογήστε και τρέξτε μόνοι σας τα παραδείγματα, πειραματιστείτε τροποποιώντας τα και λύστε τις ασκήσεις.

Θα χρειαστείτε τους δύο πιο διαδεδομένους compilers (`g++` και `clang++`), και ένα ολοκληρωμένο περιβάλλον ανάπτυξης λογισμικού (Integrated-Development-Environment, IDE).

1 Εγκατάσταση Εργαλείων

Θα χρειαστούμε 3 βασικά εργαλεία για τη μεταγλώττιση της C++ σε γλώσσα μηχανής: τους μεταγλωττιστές (compilers) g++ και `clang++`, καθώς και το `cmake`, ένα εργαλείο που οργανώνει τη μεταγλώττιση μεγαλύτερων προγραμμάτων που αποτελούνται από πολλά αρχεία. Στα επόμενα κεφάλαια θα χρησιμοποιούμε κυρίως τους δύο compilers, ενώ το `cmake` θα μας χρειαστεί όταν τα προγράμματά μας μεγαλώσουν.

Όλα τα εργαλεία εγκαθίστανται από το τερματικό (terminal) με λίγες εντολές. Ακολουθήστε την ενότητα που αντιστοιχεί στο λειτουργικό σας σύστημα και στο τέλος κάντε τον έλεγχο της εγκατάστασης.

1.1 Linux

Στο Linux αρκεί ο διαχειριστής πακέτων (package manager) της διανομής σας. Ανοίξτε ένα τερματικό και εκτελέστε τις εντολές που αντιστοιχούν στη διανομή σας:

1.1.1 Ubuntu, Debian, Linux Mint

```
sudo apt update
sudo apt install build-essential clang cmake
```

Το πακέτο `build-essential` περιέχει τον `gcc` (για την C), τον g++ και τα βασικά εργαλεία μεταγλώττισης, και το `clang` περιέχει τον `clang++`.

1.1.2 Fedora

```
sudo dnf install gcc-c++ clang cmake
```

1.1.3 Arch Linux, Manjaro

```
sudo pacman -S --needed gcc clang cmake
```

1.2 macOS

Στο macOS χρειάζεται επίσης μόνο το terminal:

1.2.1 Βήμα 1: Command Line Tools της Apple

```
xcode-select --install
```

Θα εμφανιστεί ένα παράθυρο που ζητά επιβεβαίωση· πατήστε **Install**. Με αυτόν τον τρόπο εγκαθίσταται ο `clang++` της Apple, χωρίς να χρειαστεί ολόκληρο το Xcode.

> ⚠ Ο `g++` του macOS είναι στην πραγματικότητα `clang++`
>
> Μετά το βήμα 1 υπάρχει εντολή `g++`, αλλά είναι απλώς ψευδώνυμο για τον `clang++` της Apple. Η εντολή `g++ --version` το αποκαλύπτει, αφού τυπώνει `Apple clang`. Για τον πραγματικό `g++` (GCC) χρειάζονται τα βήματα 2 και 3.

1.2.2 Βήμα 2: Homebrew

Το Homebrew είναι ο πιο διαδεδομένος διαχειριστής πακέτων για το macOS. Εγκαθίσταται με την εντολή:

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEA
```

Στο τέλος της εγκατάστασης το Homebrew τυπώνει την ενότητα **Next steps** με δύο εντολές που προσθέτουν την `brew` στο `PATH`. Αντιγράψτε τες και εκτελέστε τες. Σε υπολογιστές με επεξεργαστή Apple Silicon (M1 και νεότερους) είναι:

```
echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> ~/.zprofile
eval "$(/opt/homebrew/bin/brew shellenv)"
```

1.2.3 Βήμα 3: GCC και CMake

```
brew install gcc cmake
```

Το Homebrew εγκαθιστά τον g++ με τον αριθμό της έκδοσης στο όνομα, ώστε να μη συγκρούεται με τον g++ της Apple, π.χ. g++-16. Δείτε ποια έκδοση εγκαταστάθηκε με:

```
ls $(brew --prefix)/bin/g++-*
```

Στο macOS λοιπόν χρησιμοποιείτε g++-16 (ή όποιος αριθμός εμφανιστεί) όπου οι σημειώσεις γράφουν g++, και clang++ για τον clang.

1.3 Windows

Τα Windows δεν διαθέτουν δικό τους περιβάλλον για τους g++ και clang++. Θα τους εγκαταστήσουμε μέσω του **MSYS2**, ενός περιβάλλοντος που φέρνει στα Windows τον διαχειριστή πακέτων pacman (τον ίδιο με το Arch Linux) και, μέσω αυτού, τα εργαλεία που χρειαζόμαστε.

1.3.1 Βήμα 1: Εγκατάσταση του MSYS2

Ανοίξτε το **PowerShell** (αναζήτηση «PowerShell» στο μενού Έναρξη) και εκτελέστε:

```
winget install -e --id MSYS2.MSYS2
```

Το winget υπάρχει ήδη στα Windows 10 και 11. Το MSYS2 εγκαθίσταται στον φάκελο C:\msys64.

> **Σημείωση**
>
> Αν η εντολή winget δεν αναγνωρίζεται, ενημερώστε την εφαρμογή **App Installer** από το Microsoft Store ή κατεβάστε τον installer του MSYS2 από την επίσημη ιστοσελίδα.

1.3.2 Βήμα 2: Ενημέρωση του MSYS2

Από το μενού Έναρξη ανοίξτε το **MSYS2 UCRT64**. Προσοχή να επιλέξετε αυτό και όχι κάποιο από τα υπόλοιπα (MSYS2 MSYS, CLANG64 κ.λπ.). Στο τερματικό που ανοίγει εκτελέστε:

```
pacman -Syu
```

Πατήστε Y για επιβεβαίωση. Αν το τερματικό κλείσει μόνο του κατά την ενημέρωση, ανοίξτε ξανά το **MSYS2 UCRT64** και εκτελέστε ξανά `pacman -Syu`, μέχρι να μην υπάρχει τίποτα για ενημέρωση.

1.3.3 Βήμα 3: Εγκατάσταση των εργαλείων

Στο ίδιο τερματικό (**MSYS2 UCRT64**):

```
pacman -S --needed mingw-w64-ucrt-x86_64-gcc mingw-w64-ucrt-x86_64-clang mingw-w6
```

Πατήστε `Enter` για να επιλεγούν όλα τα πακέτα και Y για επιβεβαίωση. Το `ninja` είναι ένα γρήγορο εργαλείο build που χρησιμοποιεί το `cmake`.

1.3.4 Βήμα 4: Προσθήκη στο PATH

Για να λειτουργούν οι εντολές `g++`, `clang++` και `cmake` και εκτός του MSYS2, προσθέστε τον φάκελό τους στο PATH του χρήστη. Στο **PowerShell**:

```
[Environment]::SetEnvironmentVariable("Path", [Environment]::GetEnvironmentVariab
```

1.4 Έλεγχος της εγκατάστασης

Σε ένα **νέο** τερματικό (στα Windows, το PowerShell) εκτελέστε:

```
g++ --version
clang++ --version
cmake --version
```

Κάθε εντολή πρέπει να τυπώνει τον αριθμό της έκδοσης του αντίστοιχου εργαλείου (στο macOS, γράψτε `g++-16` αντί για `g++`).

Τέλος, δοκιμάστε να μεταγλωττίσετε ένα πρόγραμμα. Αποθηκεύστε τον παρακάτω κώδικα σε ένα αρχείο `main.cpp`:

```
#include <iostream>

int main() {
    std::cout << "All good!\n";
    return 0;
}
```

και, στον φάκελο όπου βρίσκεται το αρχείο, εκτελέστε:

```
g++ -std=c++20 -o main main.cpp
./hello
```

Αν εμφανιστεί το μήνυμα `All good!`, η εγκατάσταση ολοκληρώθηκε. Επαναλάβετε με `clang++` στη θέση του `g++` για να ελέγξετε και τον δεύτερο compiler.

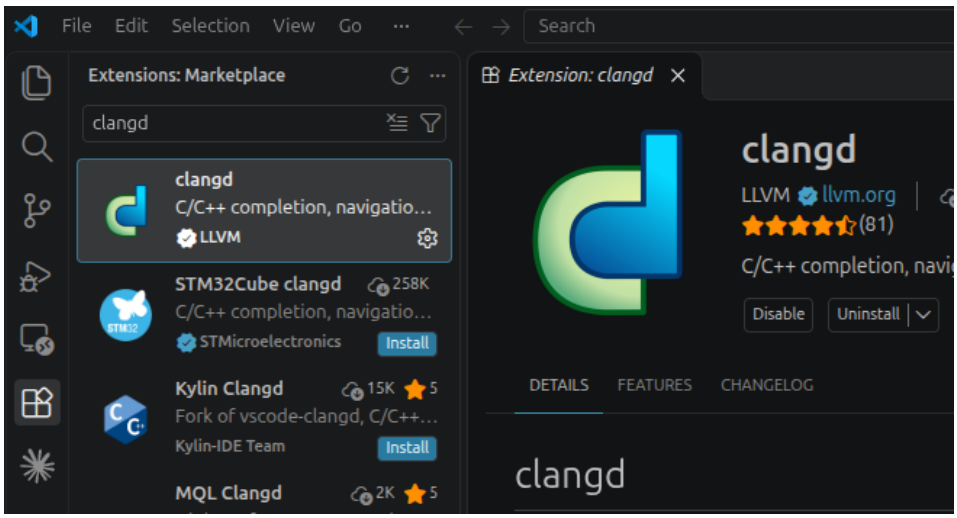
1.5 Εγκατάσταση IDE

Πριν η συγγραφή κώδικα αρχίσει να γίνεται χρησιμοποιώντας προγραμματιστικό περιβάλλον, γινόταν σε απλούς editors. Καθώς οι ανάγκες για πολύ μεγάλους κώδικες αυξάνονται, με πολλούς προγραμματιστές να δουεύουν πάνω στο ίδιο λογισμικό, η συγγραφή κώδικα γίνεται ασφαλέστερη και ευκολότερη όταν ο editor στον οποίο πληκτρολογούμε αναγνωρίζει την σύνταξη της γλώσσας και μας υποδεικνύει λάθη ή εντολές.

Ένα IDE με το οποίο οι περισσότεροι προγραμματιστές είναι εξοικειωμένοι είναι το Visual Studio Code.

1.6 Εγκατάσταση VSCode

Κατεβάστε και εγκαταστήστε το Visual Studio Code από την επίσημη ιστοσελίδα του. Μετά την εγκατάσταση, ανοίξτε τα extensions για να κατεβάσετε τον language server `clangd`:



Σχήμα 1.1: Η επέκταση `clangd` (από την LLVM) στο Extensions Marketplace του VS Code.

2 Κατασκευή προγράμματος

2.1 Compilers - Interpreters

Οι δύο βασικές κατηγορίες γλωσσών προγραμματισμού είναι οι μεταγλωττιζόμενες (compiled) και οι διερμηνευόμενες (interpreted). Οι μεν μεταφράζονται εξ-ολοκλήρου μία φορά σε γλώσσα μηχανής από , ενώ οι δε, κάθε φορά που εκτελείται ένα πρόγραμμά τους, ένα άλλο πρόγραμμα (ο διερμηνέας)

Η μεγαλύτερη ειδοποιός διαφορά μεταξύ γλωσσών όπως οι C, C++, Rust, Fortran και γλωσσών όπως οι Python και MatLab είναι η διαδικασία εκτέλεσής τους. Οι μεν ανήκουν στην κατηγορία των **μεταγλωττιζόμενων** (compiled) γλωσσών (μεταφράζονται σε γλώσσσα μηχανής), ενώ οι δε ανήκουν σε αυτή των **διερμηνευόμενων** (interpreted). Για τις interpreted, κάθε φορά που εκτελείται ένα πρόγραμμά τους, μεταφράζονται στην αρχή σε μία εσωτερική λογική δομή από τον διερμηνέα τους (interpreter), και έπειτα ο διερμηνέας εκτελεί τις εντολές. Δηλαδή απαιτείται πάντα η ύπαρξη ενός δεύτερου προγράμματος για την εκτέλεση του πρώτου. Βρισκόμαστε δηλαδή μερικά επίπεδα πιο πάνω από αυτό του επεξεργαστή.

Ουσιαστικά, όταν τρέχουμε ένα πρόγραμμα Python, η εντολή στο terminal μοιάζει κάπως έτσι:

```
python3 my_script.py
```

Με αυτή την εντολή καλούμε το πρόγραμμα `python3`, ένα εκτελέσιμο αρχείο αποθηκευμένο κάπου στον δίσκο (`.exe` στα Windows). Το αρχείο αυτό περιέχει κώδικα σε γλώσσα μηχανής, προϊόν μεταγλώττισης προγράμματος γραμμένου σε C, και αναλαμβάνει να *διερμηνεύσει* το `my_script.py`.

Με άλλα λόγια, το `my_script.py` είναι για τον διερμηνέα ό,τι ακριβώς είναι ένα αρχείο `.docx` για το Microsoft Word: δεν εκτελείται απευθείας από τον επεξεργαστή, αλλά διαβάζεται και ερμηνεύεται από ένα άλλο πρόγραμμα.

Αντίθετα, για να εκτελέσουμε ένα πρόγραμμα γραμμένο στην C++, πρώτα θα πρέπει να καλέσουμε τον compiler για να το μεταφράσει σε γλώσσα μηχανής μία και καλή (ο οποίος compiler είναι και αυτός executable όπως η διερμηνέας της Python, αλλά κάνει εντελώς άλλη δουλειά),

```
g++ -std=c++20 -o executable main.cpp
```

και μετά το μεταφρασμένο πρόγραμμα `executable` μπορεί να εκτελεστεί οποτεδήποτε:

```
./executable # Linux
```

Ωστόσο, αν αλλάξουμε μία γραμμή σε ένα πρόγραμμα Python μετά αν ξανατρέξουμε το πρόγραμμα, η αλλαγή θα φανεί κατά την εκτέλεση, ενώ αν κάνουμε το ίδιο σε μία γραμμή κώδικα γραμμένου σε C++, η αλλαγή αυτή φυσικά δεν μπορεί να γίνει εμφανής κατά την εκτέλεση του προγράμματος αν δεν κάνουμε ξανά compile το πρόγραμμα.

Πράγματι, δοκιμάστε να κάνετε compile τον παρακάτω κώδικα αφού τον αποθηκεύσετε σε ένα αρχείο `test.cpp`

```
#include <cstdio>

int main(){
    printf("Hi there.");
    return 0;
}
```

με την εντολή

```
g++ -o test test.cpp
```

και αφού εκτελέσετε το πρόγραμμα που μόλις δημιουργήθηκε δίπλα στο `test.cpp`, αλλάξτε το μήνυμα στον κώδικα και ξανατρέξτε το.

Βέβαια, να ξεκαθαριστεί πως καμία γλώσσα από μόνη της δεν είναι απαραίτητα compiled ή interpreted. Υπάρχουν interpreters για την C++, καθώς και compilers για την Python, απλά δεν χρησιμοποιούνται συχνά.

2.2 Συναρμολόγηση προγράμματος

Συνήθως, όταν αναφερόμαστε σε ένα «πρόγραμμα», εννοούμε ένα εκτελέσιμο αρχείο, το οποίο περιέχει εντολές σε γλώσσα μηχανής, δηλαδή εντολές που μπορεί να εκτελέσει απευθείας ο επεξεργαστής. Ωστόσο, ένα αρχείο που έχει προκύψει από τη μεταγλώττιση

(*compilation*) ενός προγράμματος δεν είναι απαραίτητα εκτελέσιμο από μόνο του. Μπορεί, για παράδειγμα, να περιέχει μόνο ορισμούς συναρτήσεων, οι οποίες προορίζονται να χρησιμοποιηθούν από κάποιο άλλο μέρος του προγράμματος.

Για παράδειγμα, μπορεί να έχω ένα αρχείο `tools.cpp` όπως το παρακάτω:

```
int add(int a, int b){
    // Μια συνάρτηση που παίρνει δύο αριθμούς ως παραμέτρους
    // και επιστρέφει το άθροισμά τους
    return a + b;
}

int sub(int a, int b){
    // Αντίστοιχα, επιστρέφει τη διαφορά τους
    return a - b;
}
```

Όταν μεταγλωττίσω αυτό το αρχείο, δεν υπάρχει απαραίτητα κάποιο σημείο εισόδου ώστε να ξεκινήσει να «συμβαίνει κάτι», και να μπορεί να εκτελεστεί αυτόνομα. Το αρχείο που προκύπτει, όμως, μπορεί να χρησιμοποιηθεί από το κυρίως αρχείο μου, `main.cpp`:

```
#include <cstdio>

// Δηλώνω ότι υπάρχουν αλλού οι ορισμοί των δύο συναρτήσεων,
// χωρίς να παρέχω εδώ τις πλήρεις υλοποιήσεις τους.
int add(int a, int b);
int sub(int a, int b);

// Η συνάρτηση με όνομα `main` είναι αυτή που
// ο compiler αναγνωρίζει ως κύρια, και αρχίζει να
// εκτελείται όταν ξεκινάμε το πρόγραμμα
int main(){

    int c = add(10, 20); // Αλλά πού βρίσκεται η συνάρτηση add;
    int d = sub(40, 45);

    printf("Addition: %d \n", c);     // Εκτυπώνει τον αριθμό c
    printf("Subtraction: %d \n", d); // Εκτυπώνει τον αριθμό d

    return 0;
}
```

Όταν, κατά την εκτέλεση του προγράμματος, έρθει η ώρα να υπολογιστεί η τιμή του ακεραίου c, το πρόγραμμα πρέπει να καλέσει τη συνάρτηση add. Πού βρίσκεται όμως ο κώδικας αυτής της συνάρτησης;

Ο compiler μεταφράζει τον πηγαίο κώδικα των `tools.cpp` και `main.cpp` σε κώδικα μηχανής, συνήθως μέσω της παραγωγής αντικειμενικών αρχείων (*object files*, `.o` σε Linus και `.obj` σε Windows). Οι δηλώσεις των συναρτήσεων add και sub στο `main.cpp` ενημερώνουν τον compiler για τις υπογραφές τους, αλλά δεν παρέχουν τις υλοποιήσεις τους.

Το επόμενο βήμα είναι η **σύνδεση (*linking*)**. Ο linker συνδυάζει τα αντικειμενικά αρχεία και επιλύει τις αναφορές προς συναρτήσεις που ορίζονται αλλού. Έτσι, μπορεί να αντιστοιχίσει την κλήση της add με την πραγματική υλοποίησή της στο `tools.cpp` και να δημιουργήσει το τελικό εκτελέσιμο αρχείο.

Όταν ξεκινήσει η εκτέλεση του προγράμματος, το λειτουργικό σύστημα φορτώνει στη μνήμη το εκτελέσιμο αρχείο και προετοιμάζει ό,τι απαιτείται για την εκτέλεσή του. Όταν έρθει η στιγμή να κληθεί η add, η εκτέλεση μεταφέρεται στη διεύθυνση μνήμης στην οποία βρίσκεται ο αντίστοιχος κώδικας. Ο επεξεργαστής δεν αναζητά τη συνάρτηση με το όνομά της, όπως κάνουμε εμείς όταν διαβάζουμε τον πηγαίο κώδικα. Η κλήση πραγματοποιείται μέσω των κατάλληλων εντολών μηχανής, αφού έχουν επιλυθεί οι απαραίτητες αναφορές.

Επομένως, η διαδικασία που επιτρέπει σε διαφορετικά τμήματα κώδικα να συνεργάζονται περιλαμβάνει δύο βασικά στάδια: τη **μεταγλώττιση (*compilation*)** και τη **σύνδεση (*linking*)**.

3 Εισαγωγή στην C

Πριν ξεκινήσουμε να μελετούμε την ίδια την C++, καλό είναι να κατανοήσουμε πρώτα κάποια βασικά στοιχεία της C. Η C είναι μία γλώσσα με σχετικά μικρή σύνταξη, ενώ ταυτόχρονα δίνει στον προγραμματιστή πολύ μεγάλες δυνατότητες, όπως τον έλεγχο του υπολογιστή σε πολύ χαμηλό επίπεδο και την λεπτομερή διαχείρηση μνήμης.

Στην πορεία των μαθημάτων, θα χρησιμοποιούνται διαδραστικά κομμάτια (blocks) από κώδικα, ο οποίος εκτελείται σε εξωτερικούς servers και δεν απαιτείται εγκατάσταση κάποιου προγράμματος, αν και για τις ασκήσεις η τοπική εγκατάσταση των compilers θα είναι απαραίτητη.

3.1 Ένα πρώτο πρόγραμμα

```
#include <cstdio> // Contains the `printf` function

int main() {
    printf("It's ALIIIIVEEE");
    return 0;
}
```

Παρατηρούμε διάφορα γλωσσικά στοιχεία ή λεκτικές ομάδες σε αυτό το μικρό κομμάτι κώδικα, όπως το `#include` ή το `printf`. Ο ρόλος κάθε στοιχείου θα μελετηθεί ξεχωριστά.

3.2 Τύποι αρχείων

Τα αρχεία που ο compiler διαβάζει για να συνθέσει ένα πρόγραμμα ανήκουν σε δύο κατηγορίες:

1. **Αρχεία κεφαλίδων (header files)**, με επέκταση `.h` (κυρίως για την C) ή `.hpp` (προτιμότερο για την C++, αλλά δεν έχει σημασία, είναι απλά σύμβαση), τα οποία περιέχουν δηλώσεις συναρτήσεων, σταθερών και άλλων στοιχείων που μπορούν να χρησιμοποιηθούν από άλλα αρχεία του προγράμματος. Είναι σημαντικό πως συνήθως περιέχουν την δήλωση των στοιχείων αυτών, και όχι την υλοποίησή τους (δεν είναι απαραίτητο αυτό, αλλά αργότερα θα δούμε πως αν δεν τηρούμε την σύμβαση μπορούν να δημιουργηθούν προβλήματα). Ουσιαστικά λένε στον compiler πως κάπου υπάρχουν αυτές οι μεταβλητές ή συναρτήσεις με αυτά τα ονόματα, αλλά την υλοποίησή τους θα την βρει αλλού, και απλά ο linker, μετά την μεταγλώττιση, θα πει στο τελικό πρόγραμμα πού να βρει τις διευθύνσεις μνήμης των στοιχείων αυτών όταν εκτελεστεί.
2. **Αρχεία πηγαίου κώδικα (source files)**, με επέκταση `.c` για την C, ενώ για την C++ συνηθίζεται η `.cpp` ή `.cc`. Περιέχουν την υλοποίηση των συναρτήσεων και της υπόλοιπης λειτουργικότητας του προγράμματος.

3.3 Εισαγωγή σχολίων σε κώδικα

Σε όλες τις γλώσσες προγραμματισμού είναι απαραίτητο να εξηγούμε ή να σχολιάζουμε κάποια κομμάτια του κώδικα. Αυτό σημαίνει πως θα πρέπει να γράψουμε φράσεις ή κείμενο σε φυσική γλώσσα, και ο compiler να το αγνοήσει πλήρως για να μην επηρεάσει καθόλου την διαδικασία της μετάφρασης.

Για εισαγωγή σχολίου σε μία γραμμή μόνο, γράφουμε μετά την εισαγωγή διπλής καθέτου `//`:

```
void add(int a, int b); // Δήλωση συνάρτησης. Αυτό το σχόλιο θα αγνοηθεί πλήρως α
```

Για σχόλια που χρειάζονται πάνω από μία γραμμή, γράφουμε το κείμενο ανάμεσα στο άνοιγμα σχολίου με `/*` και στο κλείσιμό του με `*/`:

```
/*
Εδώ δηλώνεται η συνάρτηση που δέχεται δύο ακεραίους ως παραμέτρους.
Το `int` στην αρχή σημαίνει απλά πως η συνάρτηση επιστρέφει επίσης ακέραιο.
*/
int add(int a, int b); /* Αυτός ο τρόπος σχολίων μπορεί να χρησιμοποιηθεί και σε
```

3.4 Εντολές Προεπεξεργαστή

Όταν πάμε να εκτελέσουμε μία εντολή μεταγλώττισης, όπως

```
g++ ...
```

πριν ξεκινήσει η διαδικασία μετάφρασης του κώδικα, ο **προ-επεξεργαστής** (*preprocessor*) καλείται πρώτος. Δεν εξετάζει καθόλου την ίδια στην σύνταξη της γλώσσας, αλλά μόνο ψάχνει τις γραμμές που ξεκινάνε με τον τελεστή #, όπως το `#include`, και τις αντικαθιστά με κώδικα, μεταβάλλοντας το κείμενο που ο compiler θα κληθεί να μεταφράσει.

Οι βασικές εντολές που μας ενδιαφέρουν για τώρα είναι οι `#include`, `#define`:

3.4.1 Η εντολή #include

Η εντολή αυτή συνοδεύεται με την διαδρομή (*path*) ενός header file (και όχι αρχεία πηγαίου κώδικα). Αν η διαδρομή είναι γραμμένη μέσα σε διπλά εισαγωγικά ως

```
#include "path/to/header.h"
```

ο προεπεξεργαστής ψάχνει το αρχείο `header.h` σε σχέση με το αρχείο στο οποίο είναι γραμμένη η εντολή. Αν για παράδειγμα βρίσκονται στον ίδιο φάκελο, τότε η σωστή εντολή είναι

```
#include "header.h"
```

ενώ αν βρίσκεται στον αμέσως πιο έξω φάκελο, η διαδρομή είναι

```
#include "../header.h"
```

Ο άλλος τρόπος να εισάγουμε την διαδρομή είναι μέσα σε <>, όπως

```
#include <header.h>
```

ή

```
#include <path/to/header.h>
```

Σε αυτή την περίπτωση, ο προεπεξεργαστής ψάχνει την διαδρομή του `header.h` όχι σε σχέση με το αρχείο στο οποίο είναι γραμμένη, αλλά σε σχέση με προεπιλεγμένους φακέλους ή φακέλους των οποίων την διαδρομή έχουμε επισημάνει στον compiler.

Η οδηγία `#include` είναι καθαρά αντικατάσταση κειμένου: ο **προεπεξεργαστής** (*preprocessor*) αντιγράφει ολόκληρο το περιεχόμενο του header file ακριβώς στο σημείο όπου βρίσκεται η οδηγία.

Η **μεταφραστική μονάδα** (*translation unit*) είναι το αποτέλεσμα αυτής της διαδικασίας. Είναι ένα αρχείο `.cpp` μαζί με όλα τα headers που περιλαμβάνει, άμεσα ή έμμεσα, αφού εκτελεστούν όλες οι οδηγίες του προεπεξεργαστή (`#include`, `#define`, `#if`, ...). Αυτό ακριβώς είναι το κείμενο που βλέπει ο **μεταγλωττιστής** (*compiler*).

Κάθε μεταφραστική μονάδα μεταγλωττίζεται **ανεξάρτητα** από τις υπόλοιπες και παράγει ένα object file (`.o`). Στη συνέχεια, ο **linker** συνδέει όλα τα object files στο τελικό εκτελέσιμο. Επομένως, όταν ο compiler επεξεργάζεται μια μεταφραστική μονάδα, δεν γνωρίζει τίποτα για τις άλλες. Γι' αυτό κάθε συνάρτηση ή στοιχείο που χρησιμοποιείται πρέπει να έχει δηλωθεί μέσα στην ίδια τη μονάδα, και αυτός είναι ο ρόλος των headers.

Συμβουλή

Μπορείτε να δείτε μια μεταφραστική μονάδα όπως τη βλέπει ο compiler, σταματώντας τη διαδικασία μετά τον προεπεξεργαστή:

```
g++ -E main.cpp -o main.ii
```

Ακόμα και ένα μικρό πρόγραμμα με `#include <iostream>` δίνει δεκάδες χιλιάδες γραμμές.

Άσκηση: Εξοικείωση με την εντολή `#include`

Στην άσκηση αυτή θα δούμε την δημιουργία συναρτήσεων με λίγα σύμβολα που δεν έχουμε δει ακόμα. Αγνοήστε τα, αρκεί απλά να ξέρετε πως απλά ορίσαμε μία συνάρτηση που τυπώνει κάτι στην κονσόλα μόλις εκτελεστεί.
Δημιουργήστε την παρακάτω δομή φακέλων:

```
greet_project/
├── main.cpp
└── include/
    ├── core/
    │   ├── greet.hpp
    │   └── shout.hpp
    └── walk.hpp
```

Μπορείτε να το κάνετε γρήγορα απλά εκτελώντας τις παρακάτω εντολές στο terminal:

3.4.2 Linux / macOS

```
mkdir -p greet_project/include/core
cd greet_project
touch main.cpp include/walk.hpp include/core/greet.hpp include/core/shout.hpp
```

3.4.3 Windows (PowerShell)

```
New-Item -ItemType Directory -Force greet_project/include/core | Out-Null
cd greet_project
New-Item -ItemType File main.cpp, include/walk.hpp, include/core/greet.hpp, inc
```

Στα αρχεία, βάλτε τα εξής περιεχόμενα.

Κατάλογος 3.1 `include/greet.hpp`

```
#include <stdio.h>

void greet() {
    printf("Hello from greet.hpp!\n");
}
```

Κατάλογος 3.2 `include/shout.hpp`

```
#include <stdio.h>

void shout() {
    printf("Shouting from shout.hpp!\n");
}
```

Κατάλογος 3.3 `include/walk.hpp`

```
#include <stdio.h>

void walk() {
    printf("Walking from walk.hpp!\n");
}
```

Κατάλογος 3.4 `main.cpp`

```
#include "include/core/greet.hpp"
#include "include/core/shout.hpp"
#include "include/walk.hpp"

int main() {
    greet();
    shout();
    walk();
    return 0;
}
```

Κάντε compile το `main.cpp` και εκτελέστε το εκτελέσιμο αρχείο:

```
g++ -o test_greet main.cpp  # compile
./test_greet                # run
```

Έπειτα, δείτε ποιά ακριβώς είναι η μεταφραστική μονάδα που βλέπει ο compiler:

```
g++ -E -o tu.ii main.cpp
```

Οι περισσότερες γραμμές στην αρχή είναι επικόλληση από το `stdio.h>`, ενώ στο τέλος είναι και ο δικός μας κώδικας.
Αν τώρα αποφασίσουμε να κάνουμε compile με την παρακάτω εντολή, η οποία λέει στον compiler να συμπεριλάβει και τον φάκελο `include` στους φακέλους που αναζητάει αρχεία, πώς θα μπορούσαμε να αντικαταστήσουμε τις 3 πρώτες γραμμές του `main.cpp`;

```
g++ -Iinclude -o test_greet main.cpp
```

> Αν βρήκατε την σωστή απάντηση και γράφετε τον κώδικα στο VSCode, ο διακομιστής γλώσσας `clangd` που είδαμε στην αρχή, αν είναι ενεργοποιημένος, θα υπογραμμίζει με κόκκινο κάποιες γραμμές γιατί παραπονιέται πως δεν υπάρχουν τα αρχεία στις τροποποιημένες γραμμές με τα `#include`. Αυτό είναι φυσιολογικό καθώς δεν γνωρίζει για την εντολή `-Iinclude` που δώσαμε στον compiler. Αν θέλουμε ο `clangd` να γνωρίζει από πριν τις παραμέτρους που δίνουμε στον `g++` ή στον `clang++`, θα πρέπει να τις δηλώσουμε στο αρχείο `CMakeLists.txt` όταν μάθουμε αργότερα πώς να φτιάχνουμε πιο οργανωμένα ένα project χρησιμοποιώντας το πρόγραμμα `cmake`.

3.4.4 Η εντολή #define

Με την εντολή `#define <name> <value>` ορίζουμε **μακροεντολές** (*macros*), ενημερώνοντας τον προεπεξεργαστή πως ό,που δει την μεταβλητή με το όνομα που δώσαμε, `<name>`, να αντικαταστήσει το κείμενό του με το κείμενο που έχουμε γράψει στο ακριβώς επόμενο πεδίο, `<value>`.

Για παράδειγμα, με την γραμμή

```
#define COUNT 10 //
```

ορίζουμε την μακροεντολή `COUNT` (είναι πολύ συνηθισμένη σύμβαση οι μακροεντολές να δηλώνονται με κεφαλαία) με τιμή 10. Είναι σημαντικό πως δεν έχει την τιμή 10 ως ακέραιο αριθμό, ο προεπεξεργαστής θα κάνει απλά αντικατάσταση κειμένου με το κείμενο “10”.

Δοκιμάστε να κάνετε compile το παρακάτω κείμενο.

```
#include <cstdio>

#define TEXT "Just some text"

int main() {
    printf(TEXT);
    return 0;
}
```

Μπορούμε να ορίσουμε macro’s που περιέχουν ολόκληρο κομμάτι κώδικα, χρησιμοποιώντας την ανάποδη κάθετο για να αλλάξουμε γραμμή και να συνεχιστεί η μακροεντολή:

```cpp
#include <cstdio>

#define TEXT "Some text"

/*
Η C και η C++ δεν απαιτούν την αλλαγή γραμμής από εντολή σε εντολή.
Θα μπορούσαμε να γράψουμε ολόκληρο λειτουργικό σύστημα σε μόνο μία τεράστια γραμμ
Αλλά στην πράξη δεν το κάνουμε ποτέ αυτό γιατί δεν θα διαβάζεται ο κώδικας,
οπότε αλλάζουμε γραμμή πολύ συχνά για να κάνουμε τη ζωή μας πιο εύκολη.

Οι μακροεντολές δεν είναι κώδικας που βλέπει ο compiler και δεν εντάσσεται σε αυτ
τον κανόνα, οπότε διαφορετικές μακροεντολές πρέπει να βρίσκονται σε διαφορετική γ
*/
#define CODE_BLOCK \  // Εδώ ξεκινάει η μακροεντολή
int main() { \
    printf(TEXT); // Εδώ τελειώνει ο ορισμός της

CODE_BLOCK // Ο preprocessor απλά κάνει επικόλληση εδώ
    return 0;
}
```

Δοκιμάστε να δείτε την μεταφραστική μονάδα (συγκεκριμένα τις τελευταίες γραμμές) που βλέπει ο compiler μετά την επεξεργασία του κώδικα από τον preprocessor, αφού αντιγράψετε τον παραπάνω κώδικα σε ένα αρχείο `pre_test.cpp`:

```
g++ -E -o tu_code.ii pre_test.cpp # Θα δημιουργηθεί το αρχείο tu_code.ii
```

Κοιτώντας τις τελευταίες γραμμές, θα δείτε πως δεν υπάρχει πουθενά η μακροεντολή `TEXT` ή `CODE_BLOCK` που ορίσαμε. Ο compiler δεν τις βλέπει ποτέ, και δεν θα γνωρίζει ποτέ για την ύπαρξή τους.